

Code of the Day

- What's the problem for the following kernel?

```
__global__ void kernel(int *input, int *output) {  
    __shared__ int shared[32];  
  
    int tid = threadIdx.x;  
    shared[tid] = input[tid];  
    __syncthreads();  
  
    shared[(tid * 2) % 32] = tid;  
    __syncthreads();  
    output[tid] = tid;  
}
```

Thread ID	Bank ID
0	$(0 * 2) \% 32 = 0$
1	$(1 * 2) \% 32 = 2$
2	$(2 * 2) \% 32 = 4$
...	...
16	$(16 * 2) \% 32 = 0$
17	$(17 * 2) \% 32 = 2$
18	$(18 * 2) \% 32 = 4$
...	...



ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Occupancy

04/07/2025

Before we get started...

- Quick overview, things discussed last time
 - Discussed GPU memory hierarchy and their optimization techniques (shared vs global memory)
- Purpose of today's lecture:
 - Atomic operations
 - Resource management; i.e., talking about “GPU occupancy”
- Miscellaneous
 - Programming assignment #6 is released – due **Friday 4/14 at 23:50 PM**
 - In-class final project presentation take place on 4/25, 4/28, 5/2 – Please sign up [here](#)

This Friday 4/11: Guest Lecture by Matthew Bruehl from Nvidia

- **GPU Programming and Optimization Techniques – plus an overview of the recent HPC job market**
- **Bio:** Matthew received a Bachelor's degree in Computer Engineering, specializing in digital design, from the University of Wisconsin-Madison in 2000. He began his career with Hewlett-Packard in Colorado, where he designed test circuits for the Floating Point Unit of the McKinley Itanium processor. Transferring to Intel in 2005, he continued work on Itanium and Xeon servers, focusing on L1D cache physical design, full-chip composition, and static timing analysis. He later enhanced BIOS testing infrastructure as part of Intel's BIOS software testing team and led the physical design tool automation flows team for a Department of Energy chip. Returning to the Xeon design team, he served as the physical design automation lead for Skylake and Sapphire Rapids chips. Transitioning to machine learning and artificial intelligence, he developed models for resource utilization and ML training in physical design and validation. After leaving Intel, he joined NVIDIA as a Logic Design ML expert, building server tools and license forecasting models. He currently leads Generative AI LLM inference and infrastructure for the ChipNemo project.



When writing software, think before you dive in...

- Design your data structures in a way that leads to advantageous memory accesses
 - Choosing a design: done before you write any line of code
 - Reflect on your algorithm: What data it needs, how much of it, where you read from/write to, how often
- A good data structure choice for parallel computing is often VERY DIFFERENT from a good data structure choice for sequential computing
 - Coalesced memory access is good for GPU but may introduces false sharing for CPU
- As we learn programming, we 100% prioritize **correctness of code**
 - Then, as one gets more sophisticated, other aspects come into play during performance optimization:
 - **speed of execution** & **convenience/productivity** & **code legacy** & **large team development** & **portability**
 - You may need to sacrifice a bit performance for the above aspects

Related to Data Organization

- Your program uses complex data constructs organized in C-structures
- For GPU computing, how is it more advantageous to store data in global memory?
 - Alternative A: as an array of structures (AoS)
 - Alternative B: as a structure of arrays (SoA)

Array of structures (AoS)

```
struct MyStruct() {  
    int x;  
    int y;  
}  
MyStruct data[100];
```

Structure of arrays (SoA)

```
int xdata[100];  
int ydata[100];
```

Answer: No universally optimal solution – it really depends on how your algorithm design in terms of memory access optimization

SoA or AoS?

- Collection of $N = 10,000$ points in 3D
- Need to compute difference between the x coordinates of a bunch of points $A_i(x_i, y_i, z_i)$:
 $|x_i - x_j|$ for $0 \leq i, j < N$
 - However, you don't use the y and z coordinates of your points
- Should I work w/ AoS or SoA?

```
int xdata[100];  
int ydata[100];  
int zdata[100];
```

vs

```
struct MyStruct() {  
    int x;  
    int y;  
    int y;  
};
```

How CPU and GPU Hide Memory Access Overhead

- CPU uses thread-level parallelism (TLP) to hide data overhead
 - Within a core, CPU uses the low-latency cache to hide memory access overhead
 - Caveat: data communication needs to find its way guaranteeing cache coherence
 - Multiple copies of the same data may exist in multiple places (caches of different cores)
- GPU hides memory latency via throughput optimization
 - Within an SM, GPU uses many threads to process many arithmetic operations
 - Caveat: data needs to find its way into the GPU global memory. Slow communication channel
 - PCIe – a good I/O bus, but not a good memory bus
 - NVLINK: a good memory bus

Atomic Operations

Coordinating Memory Operations to Avoid Data Hazards

- In **GPU computing**, accesses to shared locations (global memory & shared memory) need to be correctly orchestrated to avoid race conditions
- In many common shared memory multithreaded programming models, one uses coordination mechanisms such as **locks** to choreograph accesses to shared data
- CUDA has a scalable coordination mechanism called “**atomic memory operation**”

Example: Intra-block Race Condition

- A contrived (cooked up) example...

```
// update.cu
__global__ void update_race(int* x, int* y)
{
    int i = threadIdx.x;
    if (i < 2)
        *x += *y;
    else
        *x += 2*i;
}

// main.cpp
update_race<<<1,4>>>>(d_x, d_y);
```

Example: Intra-block Race Condition (cont'd)

- How about this?

```
// update.cu
__global__ void update_race(int* x, int* y)
{
    int i = threadIdx.x;
    if (i & 1)    // odd index
        *x += *y;
    else         // even index
        *x += 2*i;
}

// main.cpp
update_race<<<1,2>>>>(d_x, d_y);
```

Example: Inter-block Data Race

- Which one has data race?

```
// update.cu
__global__ void updateVals(int* x)
{
    int i = threadIdx.x;
    if (i == 0) *x += blockIdx.x;
}

// main.cpp
// assume d_x starting value is zero
updateVals<<<1, 5>>>(d_x);
```

```
// update.cu
__global__ void updateVals(int* x)
{
    int i = threadIdx.x;
    if (i == 0) *x += blockIdx.x;
}

// main.cpp
// assume d_x starting value is zero
updateVals<<<2, 5>>>(d_x);
```

Atomic Operations on GPU

- Atomic memory operations (atomic functions) are used to solve **access coordination** problems in parallel computing
- General concept: provide a mechanism for a thread to update a memory location such that the update appears to happen **without interruption** (atomically) with respect to all other threads – atomic operation means that operation cannot be further divided
 - Similar to `std::atomic` we talked about in CPU parallelism
- An atomic memory operation ensures two things:
 - Thing 1: Each atomic update issued is performed to completion, without any interruption
 - Thing 2: All threads can observe the outcome of any atomic update once the update completes

Atomic Functions, Example ^[1/3]

- Atomic functions are applicable to data in both **global** or **shared memory**

```
__global__ void update1(unsigned int* pX)
{
    int i = threadIdx.x;
    int j = atomicAdd(pX, i);
}
```

```
__global__ void update2()
{
    __shared__ int shm;
    int i = threadIdx.x;
    int j = atomicAdd(shm, i);
}
```

Atomic Functions [2/3]

- Synopsis of atomic function **atomicOP(a,b)** is typically (a and b below are pointers)
 - Add (add), Sub (subtract), Inc (increment), Dec (decrement)
 - And (bit-wise and), Or (bit-wise or) , Xor (bit-wise exclusive or)
 - Exch (Exchange)
 - Min (Minimum), Max (Maximum)
 - Compare-and-Swap
 - Etc.

```
t1 = *a;           // read
t2 = (*a) OP (*b); // modify
*a = t2;           // write
return t1;
```

- All statements are executed atomically without interruption by any other function/process
- Semantics and usage are very similar to `std::atomic` variables

Atomic Functions [3/3]

- Atomic functions have different latency values between shared and global memory

Memory Scope	Low Contention	High Contention
Shared Memory	~20–50 cycles	~100+ cycles
Global Memory (L2)	~400–800 cycles	~1000+ cycles
Global Memory (DRAM)	~600–1000 cycles	~1000–2000+ cycles

- Prefer running atomic operations on shared memory instead of global memory if necessary

Another Example of Atomic Operations

```
// Compute histogram of colors in an image
//
// picturePixels - pointer to picture pixels, each w/ its own color
// picturePixels[i] - takes a value between 0 and 6 (7 colors total)
// bucket-pointer to histogram bucket of size equal to # of colors
//
__global__ void histogram(int n, int* picturePixels, int* bucket)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if (i < n)
    {
        int c = picturePixels[i];
        atomicAdd(&bucket[c], 1);
    }
}
```

Performance of Atomic Operations

- While convenient to use, atomics do impact the speed of the code
 - Ex: Shared memory atomic access takes 20—100 cycles
 - Ex: Global memory atomic access takes 600—2000 cycles
- Performance is very poor when many threads attempt to perform atomic operations on the same locations
 - Bad case: all threads trying to update the same variable

Synchronization vs. Coordination

- There is a qualitative difference between a `__syncthreads()` function and an atomic operation
 - `__syncthreads()` means a *barrier*; i.e., of synchronization
 - Establishes a point in the execution of the kernel that every thread in the `**block**` needs to reach before any block thread can move beyond that point.
 - Also, all memory operations issued are seen through – no compiler reordering of instructions
- The “atomic operation” concept instead tied to the idea of coordination in relation to operations that involve `memory` transactions
 - Threads *need not* synchronize their execution, it’s only that a certain memory operation in a kernel is conducted in an atomic fashion

Resource Management Considerations

GPU is a Resourceful Device: Need Good Resource Management

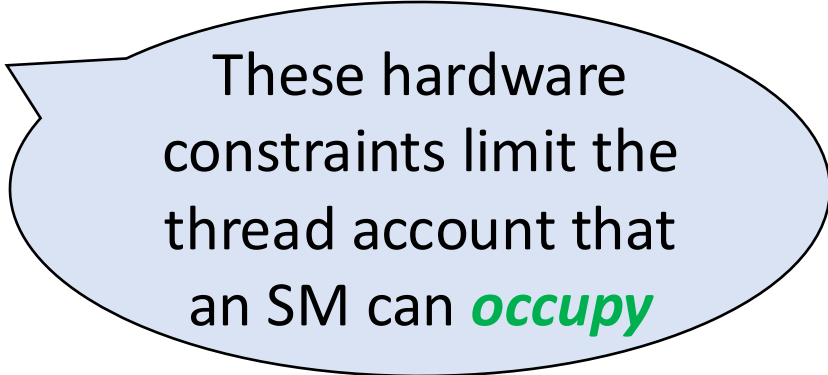
- What can we do to use the GPU at its maximum capacity?
 - “used at its maximum capacity”: SM runs the maximum number of warps it can possibly host
- The three main factors that come into play are
 - How many threads you decide to use in each block
 - $\text{threads/block} = ?$
 - What register requirements end up associated with a thread
 - $\text{registers/thread} = ?$
 - How much shared memory you assign to one block of threads
 - $\text{shMem/block} = ?$
- Beyond the three main factors above, there are some secondary factors
 - What compute capability (generation) your GPU card is
 - Your “cache vs. shared memory” choice – more shared memory, less cache benefit, and vice versa

GPU Resources Have Hard Constraints (1/2)

- Max number of **warps** that one SM can service simultaneously:
 - 48 on Fermi
 - 64 on Kepler, Maxwell, Pascal, Volta, Ampere, and Hopper
- Max number of **blocks** that one SM can process simultaneously:
 - 8 on Fermi
 - 16 on Kepler
 - 32 on Maxwell, Pascal, Volta, Ampere, and Hopper

GPU Resources Have Hard Constraints (2/2)

- The number of **32-bit registers** available on each SM is limited:
 - Roughly 48,000 on Fermi
 - Roughly 64,000 on Kepler, Maxwell, Pascal, Volta, Ampere, and Hopper
- The amount of **shared memory** available to each SM is limited
 - 64 KB on Fermi (16/48 or 48/16 configurable between L1 and shared memory)
 - 64 KB on Kepler (16/48 or 48/16 or 32/32 configurable)
 - 64 KB on Maxwell and Pascal, but not split w/ L1
 - 96 KB on Volta (this is split-able again, w/ L1 and texture)
 - Up to 164 KB on Ampere (out of 196KB, which is to be shared w/ L1)
 - Up to 256 KB on Hopper (split between L1 and ShMem)
- Note: there is also a max amount of ShMem that a block can get assigned
 - At 48 KB , sometimes 64KB, sometimes 96 KB



These hardware constraints limit the thread account that an SM can **occupy**

The Concept of Occupancy (1/2)

- A measurement of how much of a GPU's maximum thread capacity is being used
 - No directly related to performance but serves as a good proxy about resource utilization
 - Ex: Just because a program is using more GPU threads doesn't mean it is fast than that of fewer GPU threads
- Let's talk Hopper on this slide (discussion similar for Volta, Pascal, etc.)
 - On Hopper, you are allowed to serve 64 warps at the same time, with a maximum allowable number of blocks per SM being 32
 - High warp count will increase the likelihood of hiding mem access latencies with useful computation
- Occupancy examples:
 - Four blocks with 512 threads running together on one SM: 100% occupancy
 - $4 * 512 / 32 = 64$ warps
 - Four blocks of 256 threads each running on one SM: 50% occupancy
 - $4 * 256 / 32 = 32$ warps
 - How about 96 blocks with 32 threads each? – oops, can't have more than 32 blocks on a Hopper SM
 - Effectively this scenario gives you at most 50% occupancy (up to 32 blocks w/ 32 threads each)

The Concept of Occupancy (2/2)

- What prevents you from getting high occupancy?

registers	↔	per thread
shared mem	↔	per block

- Amount of shared mem demanded by each block
 - Total amount of shared memory in one SM is limited: e.g., up to 64 Kb on Pascal
- Number of registers used by each thread
 - Size of the register file (# of registers) in one SM is limited: 64K 4-byte registers on Kepler, Maxwell, Pascal, Volta, Ampere
- How do you find out how many registers and shared memory get used?
 - Solution 1: using the CUDA profiler
 - Solution 2: use the flag `-ptax-options=-v` when you compile your kernel w/ `nvcc`

Examples: Register Count (Rough Estimation)

- How many registers will the following kernel use? Assume `kernel<<<1, 32>>>`

```
__global__ void kernel(int *input, int *output) {  
    int tid = threadIdx.x;  
    output[tid] = input[tid];  
}
```

- 3 integer registers per thread => 1 block of 32 threads => $3 \times 32 = 96$ integer registers
- How about `kernel<<<2, 64>>>` ?
 - 3 integer registers per thread => 2 blocks each of 64 threads => $3 \times 2 \times 64 = 384$ integer registers

Examples: Occupancy of Fermi GPU

- Assume Fermi GPU has a limit of 48 KB Shared Memory, 48,000 registers, and 48 warps per SM
- **Example 1, Fermi:** If each of your block demands 80 KB of shared memory, the kernel will fail to launch
 - Not enough memory on the SM to run even a block
- **Example 2, Fermi:** If your blocks each uses 15 KB of shared mem, you can have up to three blocks running on one SM (there will be some shared mem that will go unused, i.e., $48 - 3 \times 15 = 3\text{KB}$ unused)
- **Example 3, Fermi:** Like Example 2 above, and you have 512 threads per block, each thread uses 30 registers. Will one SM be able to handle 2 blocks?
 - Total number of registers: $2 \times 512 \times 30 = 30,720$ out of the 48,000 are used **OK**
 - Amount of shared memory: $2 \times 15\text{K} = 30\text{ KB}$, well below 48 KB **OK**
 - Number of warps: $2 \times 512 = 1024$ threads = 32 warps < max of 48 **OK**

NVIDIA CUDA Occupancy Calculator

- There is an online “[occupancy calculator](https://xmartlabs.github.io/cuda-calculator/)” that can tell you the percentage of the HW gets utilized by your kernel
- Requires the following input
 - Compute capability
 - CUDA version
 - Threads per block
 - Registers per thread
 - Shared memory per block

CUDA Occupancy Calculator

Compute Capability version

CUDA version

Threads per block

Registers per thread

Shared memory per block

bytes

Calculate

Occupancy calculator: <https://xmartlabs.github.io/cuda-calculator/>

Example: Occupancy Study [NOTE: Specific to Fermi]

- For Matrix Multiplication example (with shared memory), should I use 8x8, 16x16, or 64x64 threads per blocks?
 - For 8x8, we have 64 threads per Block. Since each Fermi SM can manage up to 1536 resident threads, this configuration would result in $1536/64=24$ blocks. However, each Fermi SM is limited to 8 resident blocks, so only $8*64=512$ threads will go into each SM!
 - For 16x16, we have 256 threads per Block. Since a Fermi SM has 1536 resident threads, this configuration would result in $1536/256=6$ blocks, which can fit into the 8-block limit of Fermi SM
 - Next you need to see how much shared memory and how many registers get used in order to understand whether you can actually have four blocks per SM
 - 64x64 is a no starter, you can only have up to 1024 threads in a block, the tile cannot be this big

Occupancy != Performance [yet a pretty good proxy]

- Increasing occupancy does not necessarily increase performance although occupancy is a good proxy for measuring GPU performance
 - What controls the damage is the Instruction Level Parallelism (ILP) that the compiler can capitalize on (better efficiency in pipelining, instruction reordering, etc.)

HOWEVER,

- Low-occupancy multiprocessors are likely to have a hard time when it comes to hiding latency on memory-bound kernels
 - This latency hiding draws on Thread Level Parallelism (TLP); i.e., having enough threads (warps, that is) that are ready for execution
 - Basically, it means some warps are idle which could otherwise be dispatched to do useful jobs

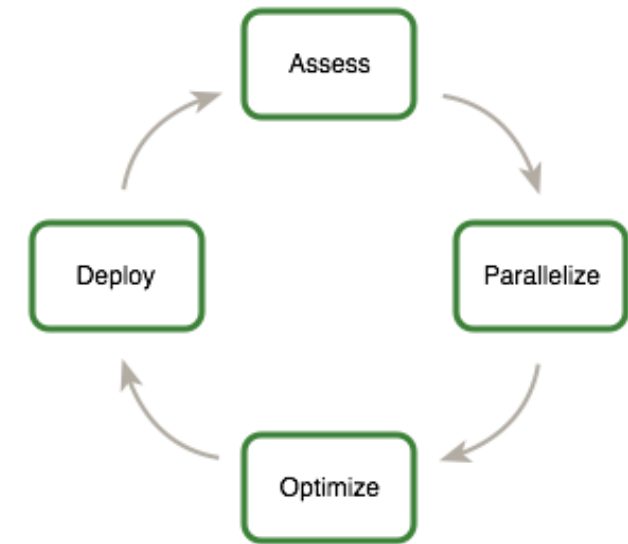
How Do We Incorporate these Hardware Factors?

- Parameterization helps adapt your kernel implementation to different GPUs
 - For the purpose of performance portability
- GPUs of different generation (compute capabilities) vary in many ways
 - # of SMs
 - Memory bandwidth
 - Shared memory size
 - Register file size
 - Max. threads per block
 - Max. number of warps per SM
- You can even make apps self-tuning – an open and active research area
 - Ex: leverage machine learning to automatically “recommend” a good parameter combination

CUDA Optimization Rules of Thumb

Writing CUDA Software: High-Priority Recommendations

1. To get the highest benefit from CUDA, focus first on finding ways to **parallelize sequential** code through exploring data-parallel patterns
2. **Minimize data transfer** between the host and the device, even if it means running some kernels on the device that do not show performance gains when compared with running them on the host CPU
3. Strive to have **aligned and coalesced global memory accesses**. Design your implementation such that global memory accesses are coalesced for that part of the red-hot parts of the code
4. **Minimize the use of global memory**. Prefer **shared memory** access where possible (consider tiling as a design solution)



Very good resource: <http://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#abstract>

Writing CUDA Software: Medium-Priority Recommendations

1. Accesses to shared memory should be designed to avoid serializing requests due to **bank conflicts**
2. Strive for sufficient occupancy
3. Keep the number of threads per block a **multiple of 32** to avoid wasted lanes
4. Use the **fast math library** whenever speed is very important, and you can live with a tiny loss of accuracy
5. **Avoid thread divergence**